

#3. Deep Architectures for Tabular Data

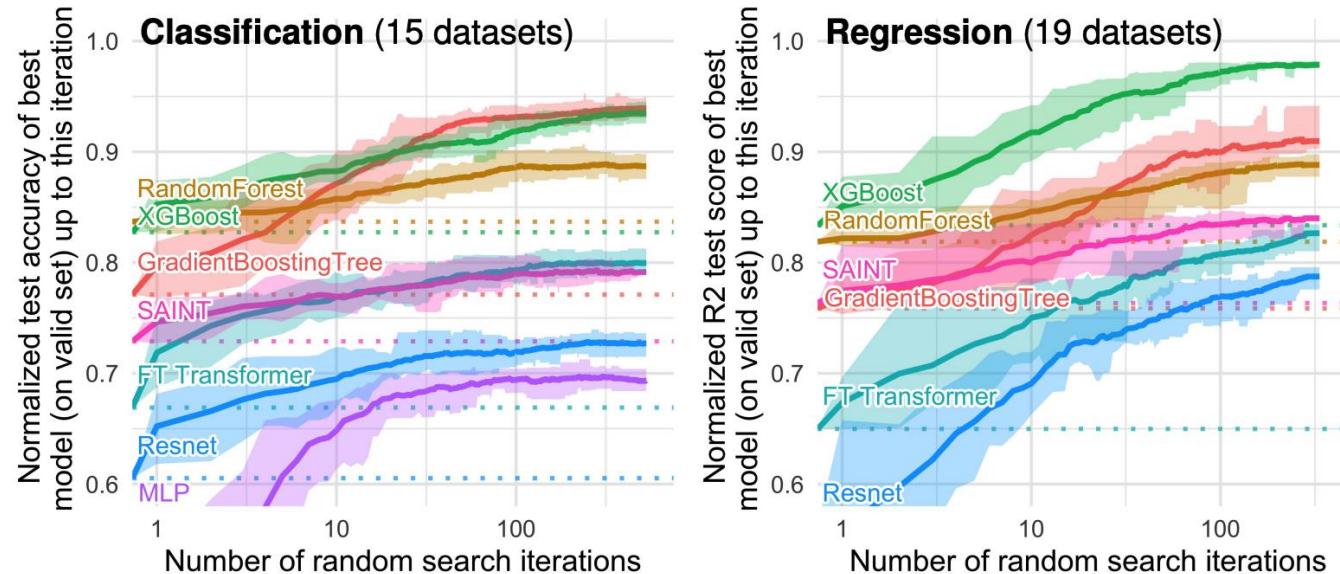
Instructor: **Hankook Lee** @ Efficient Learning Lab., Sungkyunkwan University

Lecture Overview

- This lecture covers **deep neural networks** for tabular data
 - **Why deep learning?**
 - **MLP-based architectures:** How can simple MLPs be made truly competitive?
 - ResNet-MLP (2021), MLP-PLR (2022), RealMLP (2024), TabR (2024), ModernNCA (2025), TabM (2025)
 - **Attention-based architectures:** How to model interactions among features and samples?
 - TabTransformer (2020), FT-Transformer (2021), NPT (2021), SAINT (2021), T2G-Former (2023), ExcelFormer (2024), AMFormer (2024)

Tree-based Models Are Still Strong

- Tree-based models remain highly competitive for tabular prediction



Empirical Findings:

- NNs are biased to overly smooth solutions
- Uninformative features affect more MLP-like NNs
- Data are non invariant by rotation

Then, Why Deep Learning?

- **Learnable Representations**

- Neural models can learn embeddings for categorical and numerical features, **capturing richer similarity and structure than raw values** alone

- **End-to-end Feature Interactions**

- MLPs and Transformers can jointly learn **complex non-linear interactions among features**, rather than constructing them via sequential tree splits

- **Extensibility Beyond Standard Prediction**

- Neural architectures naturally **extend to pretraining, retrieval, multitask learning, and multimodal modeling** with text, images, or time series

Simple MLP Baselines

- Build **plain & ResNet-like MLPs** on a tabular row $\mathbf{x} \in \mathbb{R}^d$
 - Treats every numerical column as a single scalar feeding a dense layer
 - Categorical features need one-hot or learned embeddings

Plain MLP

$$\begin{aligned}\text{MLP}(x) &= \text{Linear}(\text{MLPBlock}(\dots(\text{MLPBlock}(x)))) \\ \text{MLPBlock}(x) &= \text{Dropout}(\text{ReLU}(\text{Linear}(x)))\end{aligned}$$

ResNet-like MLP

$$\begin{aligned}\text{ResNet}(x) &= \text{Prediction}(\text{ResNetBlock}(\dots(\text{ResNetBlock}(\text{Linear}(x))))) \\ \text{ResNetBlock}(x) &= x + \text{Dropout}(\text{Linear}(\text{Dropout}(\text{ReLU}(\text{Linear}(\text{BatchNorm}(x))))) \\ \text{Prediction}(x) &= \text{Linear}(\text{ReLU}(\text{BatchNorm}(x)))\end{aligned}$$

Simple MLP Baselines

- Build **plain & ResNet-like MLPs** on a tabular row $\mathbf{x} \in \mathbb{R}^d$
 - Treats every numerical column as a single scalar feeding a dense layer
 - Categorical features need one-hot or learned embeddings

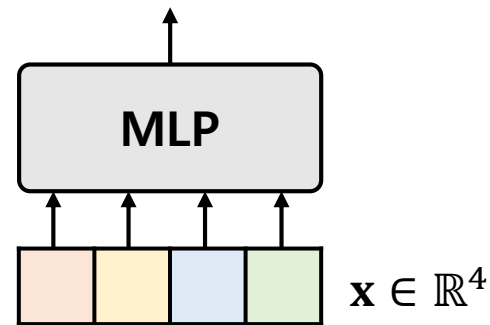
	CA ↓	AD ↑	HE ↑	JA ↑	HI ↑	AL ↑	EP ↑	YE ↓	CO ↑	YA ↓	MI ↓	rank (std)
TabNet	0.510	0.850	0.378	0.723	0.719	0.954	0.8896	8.909	0.957	0.823	0.751	7.5 (2.0)
SNN	0.493	0.854	0.373	0.719	0.722	0.954	0.8975	8.895	0.961	0.761	0.751	6.4 (1.4)
AutoInt	0.474	0.859	0.372	0.721	0.725	0.945	0.8949	8.882	0.934	0.768	0.750	5.7 (2.3)
GrowNet	0.487	0.857	–	–	0.722	–	0.8970	8.827	–	0.765	0.751	5.7 (2.2)
MLP	0.499	0.852	0.383	0.719	0.723	0.954	0.8977	8.853	0.962	0.757	0.747	4.8 (1.9)
DCN2	0.484	0.853	0.385	0.716	0.723	0.955	0.8977	8.890	0.965	0.757	0.749	4.7 (2.0)
NODE	0.464	0.858	0.359	0.727	0.726	0.918	0.8958	8.784	0.958	0.753	0.745	3.9 (2.8)
ResNet	0.486	0.854	0.396	0.728	0.727	0.963	0.8969	8.846	0.964	0.757	0.748	3.3 (1.8)
FT-T	0.459	0.859	0.391	0.732	0.729	0.960	0.8982	8.855	0.970	0.756	0.746	1.8 (1.2)

Simple MLP Baselines

- Build **plain & ResNet-like MLPs** on a tabular row $\mathbf{x} \in \mathbb{R}^d$
 - Treats every numerical column as a single scalar feeding a dense layer
 - Categorical features need one-hot or learned embeddings
- **Main takeaways**
 - MLP is still a good sanity check
 - **ResNet turns out to be an effective baseline** that none of the competitors can consistently outperform
 - **Tuning makes simple models such as MLP and ResNet competitive**
 - Recommend tuning baselines when possible
 - There is still no universal solution among DL models and GBDT

Better Embedding for Numerical Features

- **A simple way:** taking each numerical feature as a single scalar input

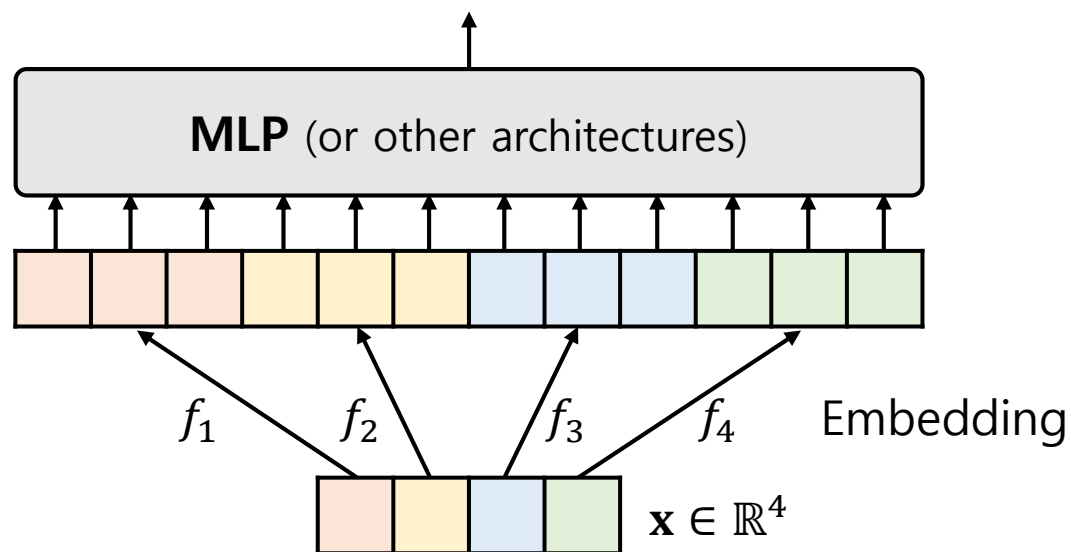


Limitations:

- Each numerical feature is represented by only one scalar value
- The model must learn complex patterns only after raw features are mixed
 - Threshold-like patterns
 - Local variations across value ranges
 - Irregular relationships with the target

Better Embedding for Numerical Features

- **An alternative:** taking each numerical feature as a single vector
 - $\mathbf{z}_i = f_i(x_i) \in \mathbb{R}^{d_i}$, f_i is the embedding function for the i-th numerical feature



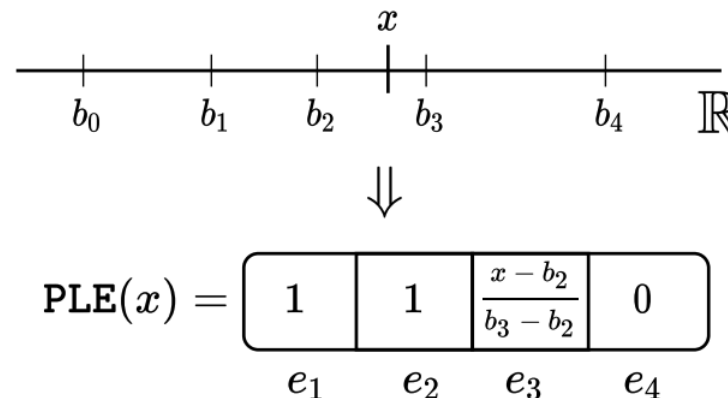
- Each numerical feature is embedded independently
- **(Q)** Which embedding function is effective for numerical features?

Better Embedding for Numerical Features

1. Piecewise Linear Encoding (PLE)

$$\text{PLE}(x) = [e_1, \dots, e_T] \in \mathbb{R}^T$$

$$e_t = \begin{cases} 0, & x < b_{t-1} \text{ AND } t > 1 \\ 1, & x \geq b_t \text{ AND } t < T \\ \frac{x - b_{t-1}}{b_t - b_{t-1}}, & \text{otherwise} \end{cases}$$



- Bins can be constructed from quantiles or target-aware splits

2. Periodic Activation Functions

$$f_i(x) = \text{Periodic}(x) = \text{concat}[\sin(v), \cos(v)], \quad v = [2\pi c_1 x, \dots, 2\pi c_k x]$$

- c_i are trainable parameters initialized from $\mathcal{N}(0, \sigma)$

Better Embedding for Numerical Features

- Feature-wise embedding consistently improves performance

	GE $\uparrow$	CH $\uparrow$	CA $\downarrow$	HO $\downarrow$	AD $\uparrow$	OT $\uparrow$	HI $\uparrow$	FB $\downarrow$	SA $\uparrow$	CO $\uparrow$	MI $\downarrow$
MLP	0.632	0.856	0.495	3.204	0.854	0.818	0.720	5.686	0.912	0.964	0.747
MLP-L	0.639	0.861	0.475	3.123	0.856	0.820	0.723	5.684	0.916	0.963	0.748
MLP-LR	0.642	0.860	0.471	3.084	0.857	0.819	0.726	5.625	0.923	0.963	0.746

	GE $\uparrow$	CH $\uparrow$	CA $\downarrow$	HO $\downarrow$	AD $\uparrow$	OT $\uparrow$	HI $\uparrow$	FB $\downarrow$	SA $\uparrow$	CO $\uparrow$	MI $\downarrow$
MLP	0.632	0.856	0.495	3.204	0.854	0.818	0.720	5.686	0.912	0.964	0.747
MLP-Q	0.653	0.854	0.464	3.163	0.859	0.816	0.721	5.766	0.922	0.968	0.750
MLP-T	0.647	0.861	0.447	3.149	0.864	0.821	0.720	5.577	0.923	0.967	0.749
MLP-Q-LR	0.646	0.857	0.455	3.184	0.863	0.811	0.720	5.394	0.923	0.969	0.747
MLP-T-LR	0.640	0.861	0.439	3.207	0.868	0.818	0.724	5.508	0.924	0.968	0.747

	GE $\uparrow$	CH $\uparrow$	CA $\downarrow$	HO $\downarrow$	AD $\uparrow$	OT $\uparrow$	HI $\uparrow$	FB $\downarrow$	SA $\uparrow$	CO $\uparrow$	MI $\downarrow$
MLP	0.632	0.856	0.495	3.204	0.854	0.818	0.720	5.686	0.912	0.964	0.747
MLP-P	0.631	0.860	0.489	3.129	0.869	0.807	0.723	5.845	0.923	0.968	0.747
MLP-PL	0.641	0.859	0.467	3.113	0.868	0.819	0.727	5.530	0.924	0.969	0.746
MLP-PLR	0.674	0.857	0.467	3.050	0.870	0.819	0.728	5.525	0.924	0.970	0.746

Name	Embedding function (f_i)
L	Linear
LR	$\text{ReLU} \circ \text{Linear}$
Q	PLE_q
Q-L	$\text{Linear} \circ \text{PLE}_q$
Q-LR	$\text{ReLU} \circ \text{Linear} \circ \text{PLE}_q$
T	PLE_t
T-L	$\text{Linear} \circ \text{PLE}_t$
T-LR	$\text{ReLU} \circ \text{Linear} \circ \text{PLE}_t$
P	Periodic
PL	$\text{Linear} \circ \text{Periodic}$
PLR	$\text{ReLU} \circ \text{Linear} \circ \text{Periodic}$

Better Embedding for Numerical Features

- MLP with better embeddings outperforms tree-based and Transformer-based models

	GE $\uparrow$	CH $\uparrow$	CA $\downarrow$	HO $\downarrow$	AD $\uparrow$	OT $\uparrow$	HI $\uparrow$	FB $\downarrow$	SA $\uparrow$	CO $\uparrow$	MI $\downarrow$	Avg. Rank
CatBoost	0.692	0.861	0.430	3.093	0.873	0.825	0.727	5.226	0.924	0.967	0.741	3.6 ± 2.9
XGBoost	0.683	0.859	0.434	3.152	0.875	0.827	0.726	5.338	0.919	0.969	0.742	4.6 ± 2.7
MLP	0.665	0.856	0.486	3.109	0.856	0.822	0.727	5.616	0.913	0.968	0.746	8.5 ± 2.6
MLP-LR	0.679	0.861	0.463	3.012	0.859	0.826	0.731	5.477	0.924	0.972	0.744	5.5 ± 2.7
MLP-Q-LR	0.682	0.859	0.433	3.080	0.867	0.818	0.724	5.144	0.924	0.974	0.745	5.1 ± 1.9
MLP-T-LR	0.673	0.861	0.435	3.099	0.870	0.821	0.727	5.409	0.924	0.973	0.746	5.1 ± 1.7
MLP-PLR	0.700	0.858	0.453	2.975	0.874	0.830	0.734	5.388	0.924	0.975	0.743	3.0 ± 2.4
ResNet	0.690	0.861	0.483	3.081	0.856	0.821	0.734	5.482	0.918	0.968	0.745	6.7 ± 3.3
ResNet-LR	0.672	0.862	0.450	2.992	0.859	0.822	0.733	5.415	0.923	0.971	0.743	5.6 ± 2.7
ResNet-Q-LR	0.674	0.859	0.427	3.066	0.868	0.815	0.729	5.309	0.923	0.976	0.746	4.7 ± 2.0
ResNet-T-LR	0.683	0.862	0.425	3.030	0.872	0.822	0.731	5.471	0.923	0.975	0.744	4.1 ± 1.9
ResNet-PLR	0.691	0.861	0.443	3.040	0.874	0.825	0.734	5.400	0.924	0.975	0.743	3.2 ± 1.3

- **Main Takeaway:**

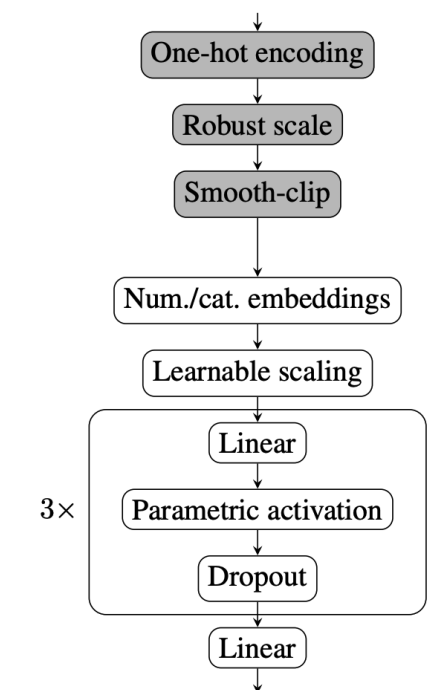
- For tabular deep learning, how numerical features are represented can matter as much as the choice of backbone

A Better Training Recipe for MLPs

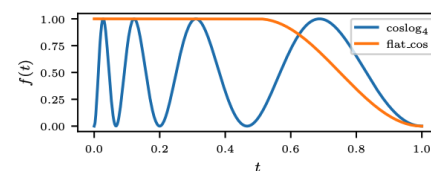
- **Observation:** Many tabular DNN papers compare against weak or insufficiently tuned MLP baselines
- **Question:** Are MLPs really weak for tabular data?
- Holzmüller et al. (2024) argue that a simple MLP can be strong with:
 - Robust preprocessing
 - Better numerical and categorical feature representations
 - Improved initialization and training schedules
 - Meta-tuned default hyperparameters

A Better Training Recipe for MLPs

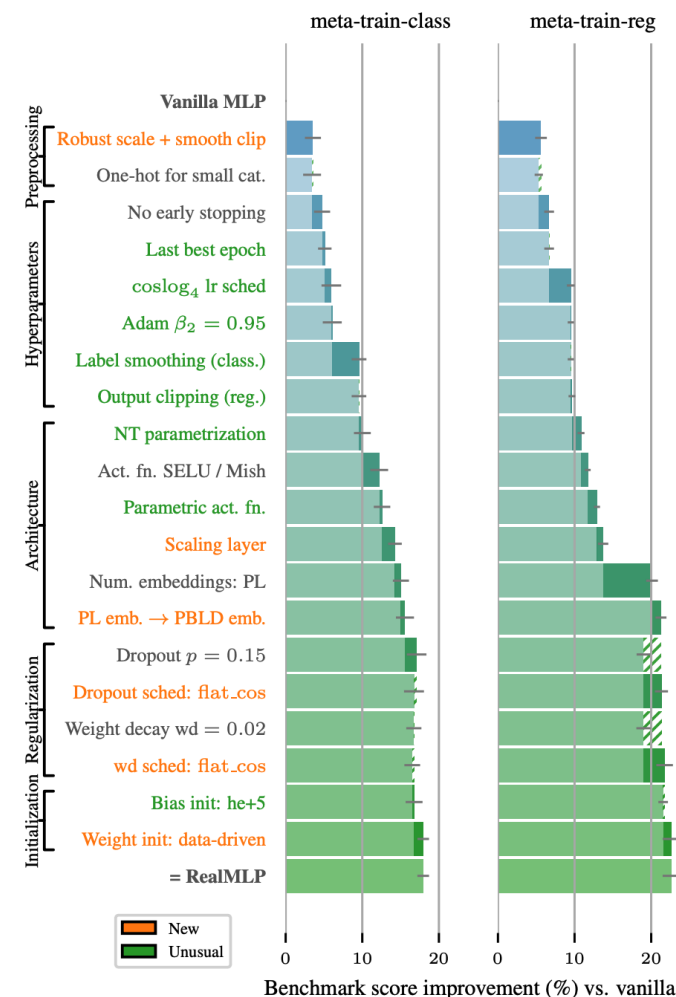
- **RealMLP** improves a vanilla MLP:
 - Robust preprocessing
 - Numerical and categorical embeddings
 - Learnable feature scaling
 - Improved activations, initialization, and regularization
- A meta-tuned **default recipe**
 - Tune the default recipe on many **meta-train datasets**
 - Apply it to new datasets **without expensive per-dataset HPO**



(a) Preprocessing and NN architecture for RealMLP-TD.



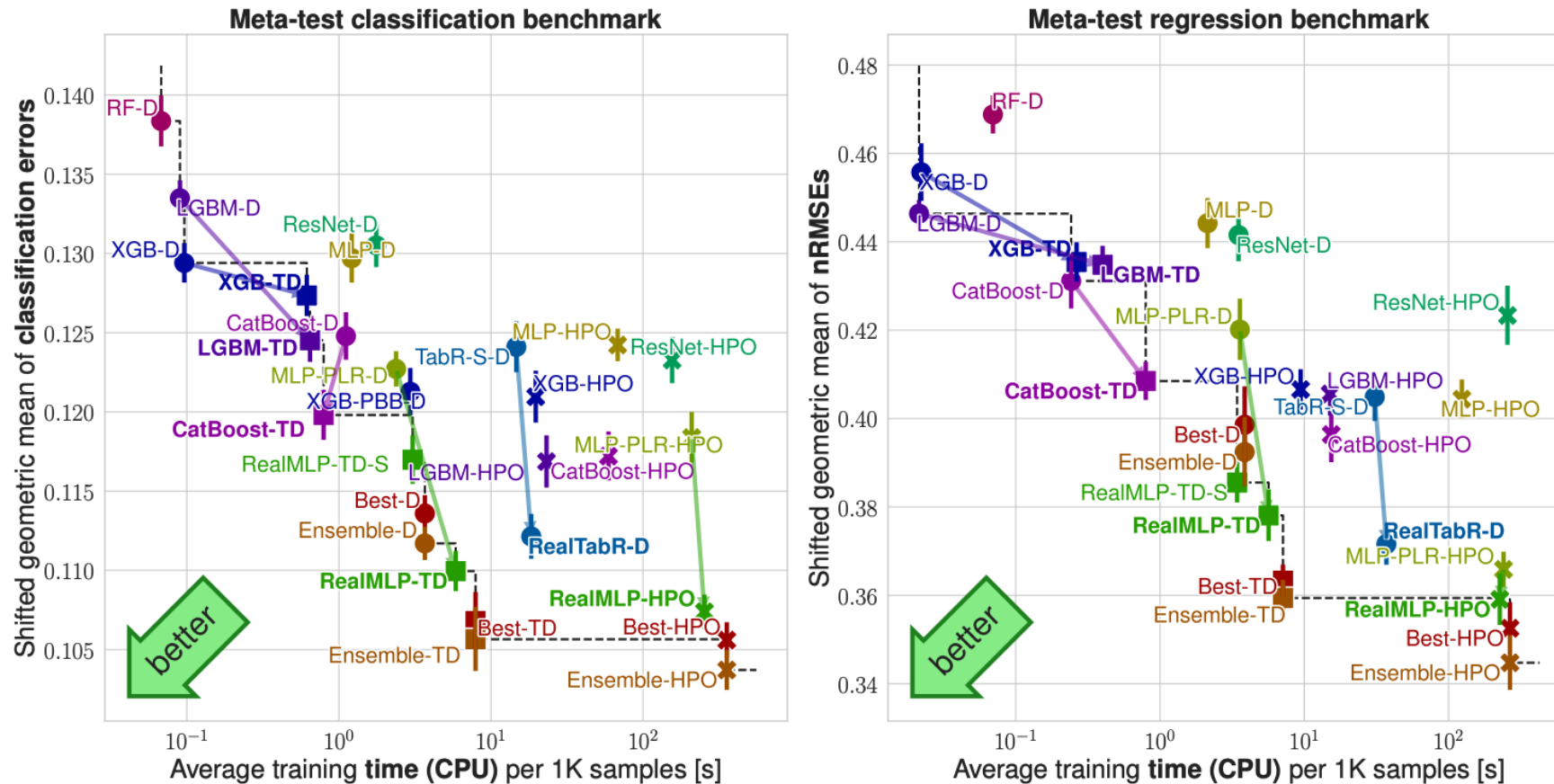
(b) The coslog_4 and flat_cos schedules.



(c) From a vanilla MLP to RealMLP-TD.

A Better Training Recipe for MLPs

- A carefully designed, meta-tuned MLP achieves a strong accuracy-efficiency trade-off on unseen tabular datasets

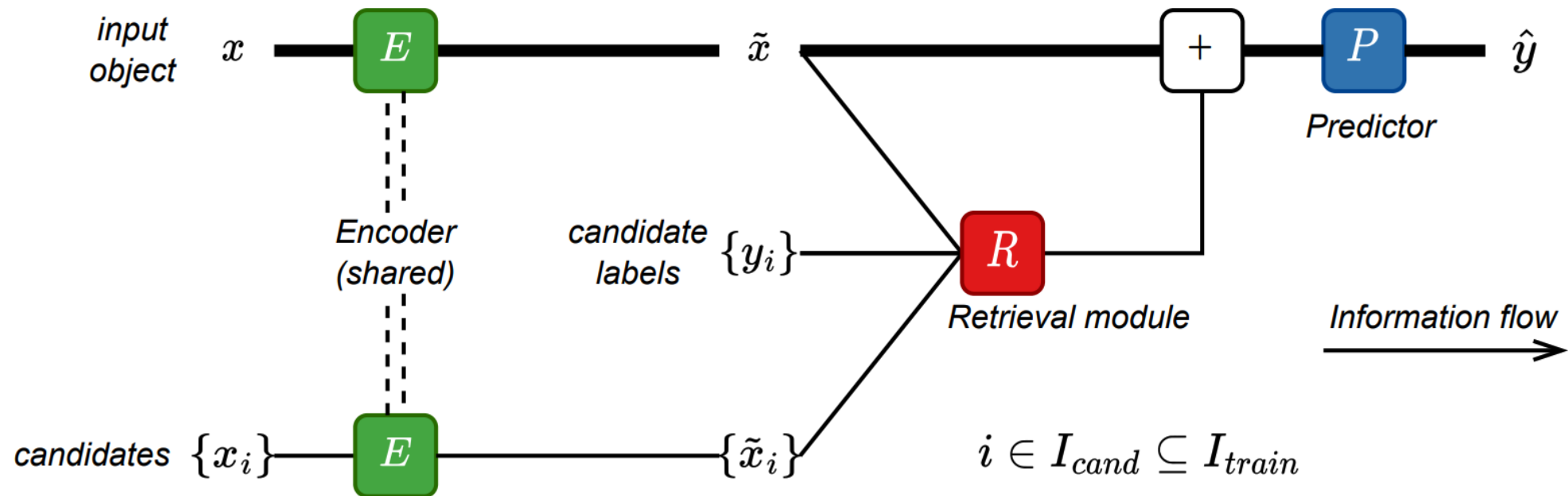


Deep Learning from Nearest Neighbors

- **Motivation:** Similar rows can provide useful information for prediction
 - KNN uses nearby training samples directly
 - Neural networks usually compress training data into model parameters
- **Question:** Can we combine neural representations with nearest-neighbor reasoning?
- **Two recent directions**
 - **TabR:** Retrieve similar training samples and use them inside a neural network
 - **ModernNCA:** Learn an embedding space where soft nearest-neighbor prediction works well

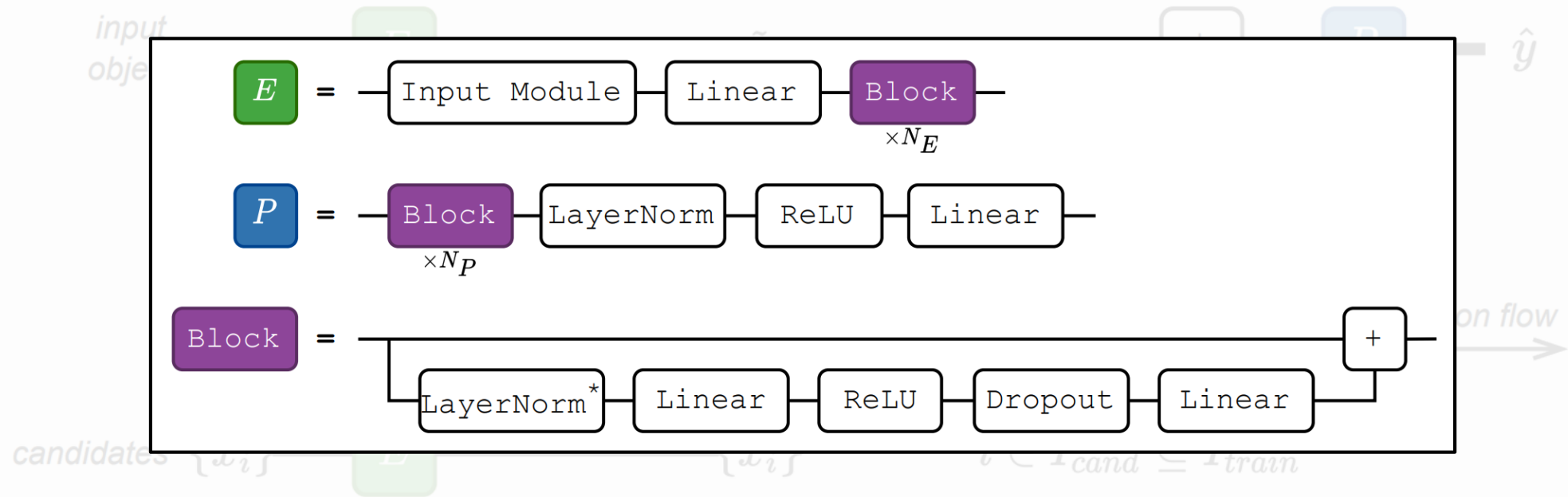
Retrieval-Augmented Tabular Prediction

- **TabR** is a MLP **with a kNN-like retrieval** module
 - Retrieve neighbors on the embedding space from the candidate set
 - Use both neighbor representations and neighbor labels
 - Add retrieved information to the target representation before prediction



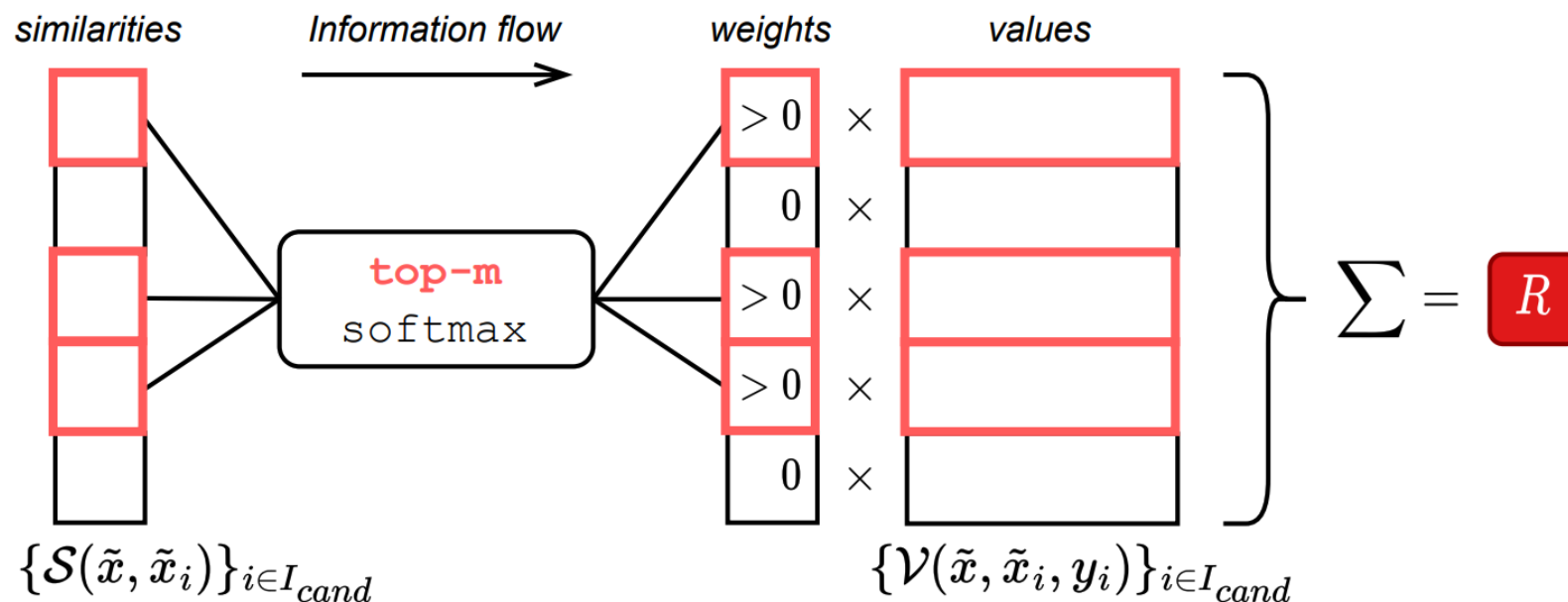
Retrieval-Augmented Tabular Prediction

- **TabR** is a MLP **with a kNN-like retrieval** module
 - Retrieve neighbors on the embedding space from the candidate set
 - Use both neighbor representations and neighbor labels
 - Add retrieved information to the target representation before prediction



Retrieval-Augmented Tabular Prediction

- The retrieval module works as attention:
 - Retrieve **only the top-m relevant neighbors**
 - Use context labels and the difference between target and neighbor samples



$$k = W_K(\tilde{x}), k_i = W_K(\tilde{x}_i) \quad \mathcal{S}(\tilde{x}, \tilde{x}_i) = -\|k - k_i\|^2 \quad \mathcal{V}(\tilde{x}, \tilde{x}_i, y_i) = W_Y(y_i) + T(k - k_i)$$

Retrieval-Augmented Tabular Prediction

- **TabR** (1) becomes a new strong deep learning solution and (2) demonstrates a good potential of the retrieval-based approach

CH ↑ CA ↓ HO ↓ AD ↑ DI ↓ OT ↑ HI ↑ BL ↓ WE ↓ CO ↑ MI ↓ Avg. Rank																	
Tuned hyperparameters																	
XGBoost	0.861	0.432	3.164	0.872	0.136	0.832	0.726	0.680	1.769	0.971	0.741	2.5 ± 0.9					
CatBoost	0.859	0.426	3.106	0.872	0.133	0.827	0.727	0.681	1.773	0.969	0.741	2.5 ± 1.1					
LightGBM	0.860	0.434	3.167	0.872	0.136	0.832	0.726	0.679	1.761	0.971	0.741	2.4 ± 0.9					
TabR	0.865	0.391	3.025	0.872	0.131	0.831	0.733	0.674	1.661	0.977	0.748	1.3 ± 0.9					
Default hyperparameters																	
XGBoost	0.856	0.471	3.368	0.871	0.143	0.817	0.716	0.683	1.920	0.966	0.750	3.4 ± 0.9					
CatBoost	0.861	0.432	3.108	0.874	0.132	0.822	0.726	0.684	1.886	0.924	0.744	2.1 ± 0.8					
LightGBM	0.856	0.449	3.222	0.869	0.137	0.826	0.720	0.681	1.817	0.899	0.744	2.5 ± 0.9					
TabR-S	0.864	0.398	2.971	0.859	0.131	0.824	0.724	0.688	1.721	0.974	0.752	2.0 ± 1.3					
DKE (Wilson et al., 2016)																	
ANP (Kim et al., 2019)						–	0.521	3.423	–	0.147	–	–	6.2 ± 0.5				
SAINT (Somepalli et al., 2021)						–	0.472	3.162	–	0.140	–	–	4.6 ± 2.5				
NPT (Kossen et al., 2021)						0.860	0.468	3.242	0.860	0.137	0.812	0.724	0.693	1.933	0.964	0.763	3.8 ± 1.5
MLP						0.858	0.474	3.175	0.853	0.138	0.815	0.721	0.692	1.947	0.966	0.753	3.6 ± 1.0
MLP-PLR						0.854	0.499	3.112	0.853	0.140	0.816	0.719	0.697	1.905	0.963	0.748	3.7 ± 1.3
TabR-S						0.860	0.476	3.056	0.870	0.134	0.819	0.729	0.687	1.860	0.970	0.744	2.0 ± 1.0
TabR						0.860	0.403	3.067	0.865	0.133	0.818	0.722	0.690	1.747	0.973	0.750	1.9 ± 0.7
						0.862	0.400	3.105	0.870	0.133	0.825	0.729	0.676	1.690	0.976	0.750	1.3 ± 0.6

ModernNCA

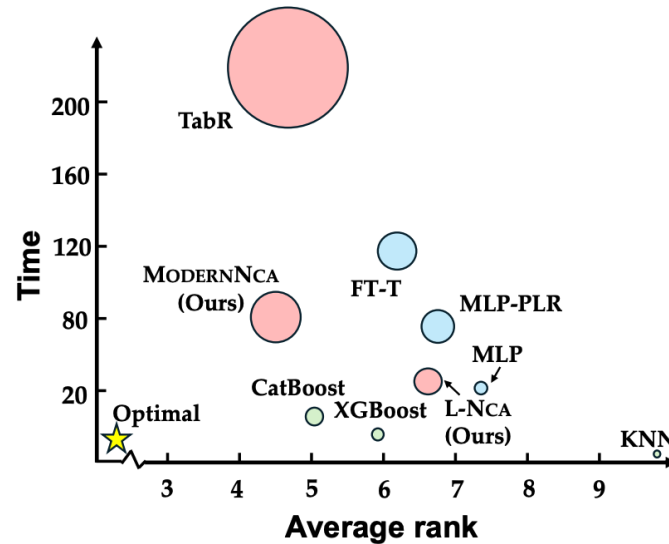
- **ModernNCA** revisits Neighborhood Components Analysis (NCA) with a **learnable embedding function** (i.e., neural network) ϕ

$$\hat{y}_i = \sum_{(\mathbf{x}_j, y_j) \in \mathcal{D}} \frac{\exp(-\text{dist}^2(\phi(\mathbf{x}_i), \phi(\mathbf{x}_j)))}{\sum_{(\mathbf{x}_l, y_l) \in \mathcal{D}, \mathbf{x}_l \neq \mathbf{x}_i} \exp(-\text{dist}^2(\phi(\mathbf{x}_i), \phi(\mathbf{x}_l)))} y_j$$

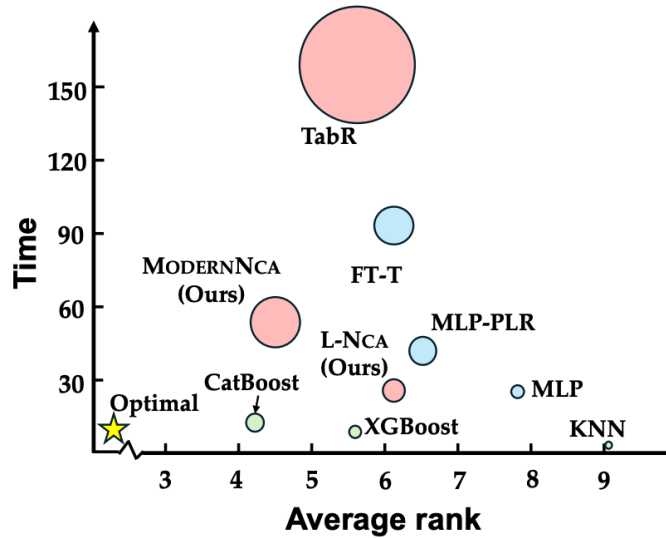
- This is also known as soft nearest neighbor (soft-NN)
- Why "modern"?
 - Better embedding with a neural network and PLR-style numerical encoding
 - Better optimization
 - Efficient neighborhood training with Stochastic Neighborhood Sampling (SNS)
 - Simpler and faster than more complex retrieval-based models such as TabR

TabR vs ModernNCA

Method	Key Idea	Prediction Mechanism
TabR	Add retrieved neighbor information inside a neural network	Encoder + retrieval module + predictor
ModernNCA	Learn a space where soft nearest-neighbor prediction works	Weighted average of neighbor labels



(a) Classification



(b) Regression

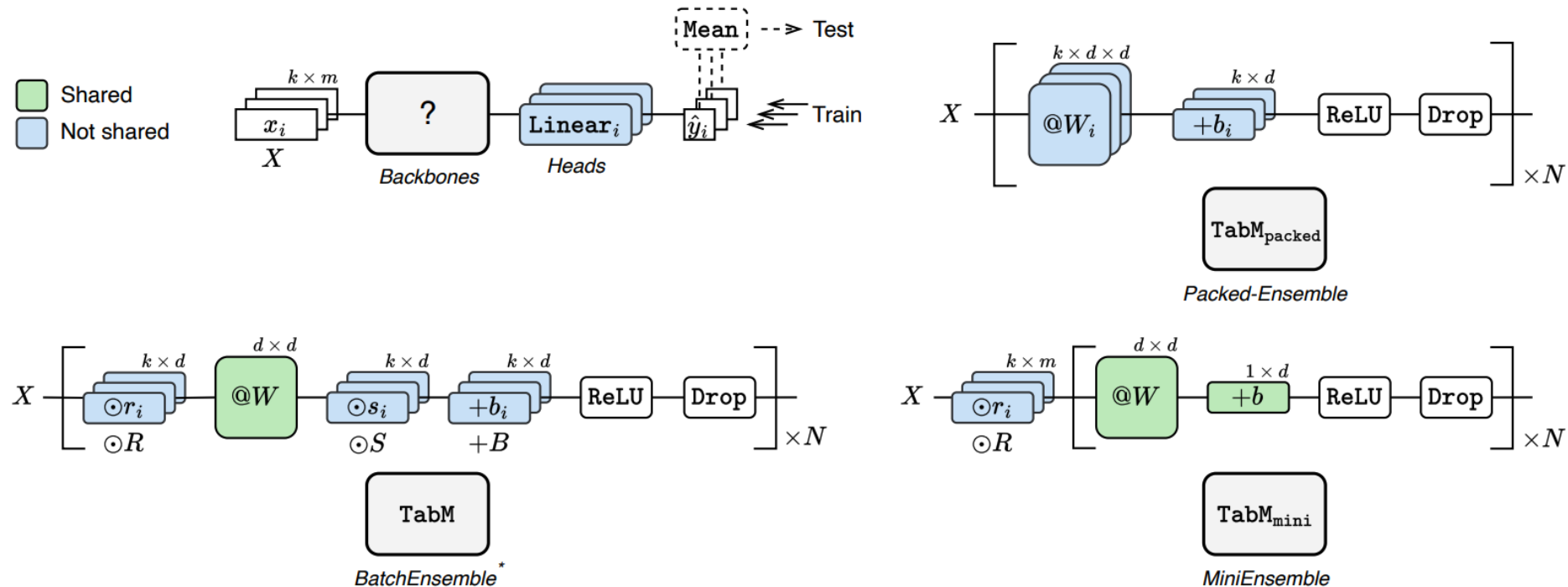
Takeaway: Neighborhood information is a powerful inductive bias for tabular deep learning.

TabM: MLP with Ensemble

- **Observation:** Most tabular DL methods have focused on building a stronger **single predictor** through:
 - Better feature representations
 - Better training recipes
 - Attention or retrieval mechanisms
- **Ensembling has been largely overlooked in tabular deep learning!**
- **Question:** Can we efficiently train an **ensemble of neural predictors**?
 - Is ensembling effective for tabular prediction?
 - Can we obtain ensemble benefits through parameter sharing?

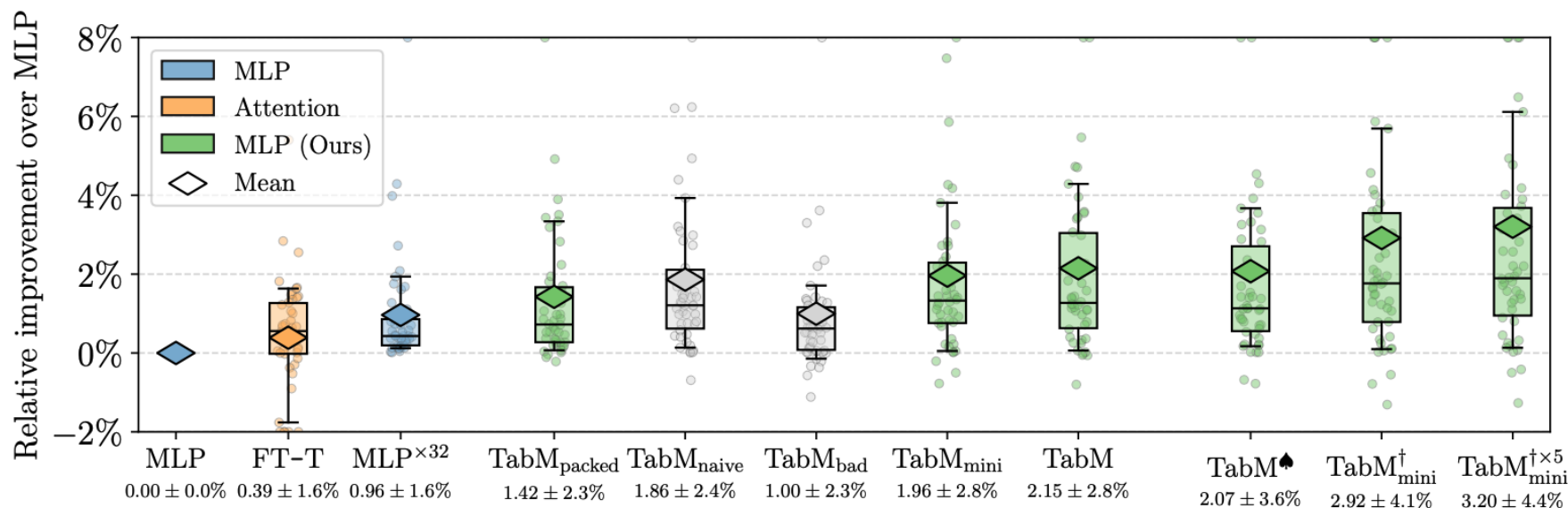
TabM: MLP with Ensemble

- How to build an ensemble?
 - **Deep Ensemble**: Train multiple MLPs independently
 - **Packed-Ensemble**: Pack multiple MLPs into one and train them jointly
 - **TabM & TabM_{mini}**: Share most parameters across ensemble members



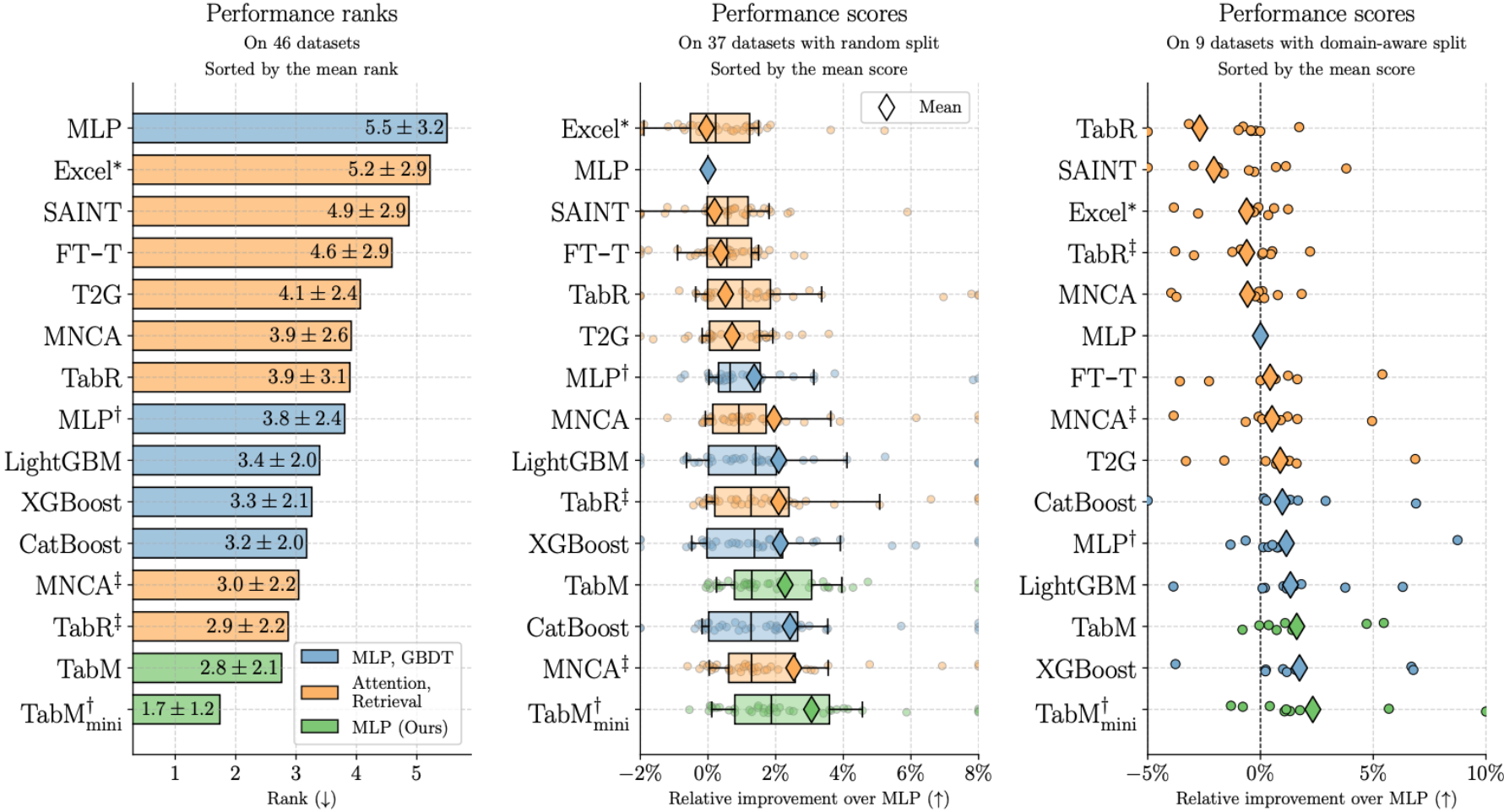
TabM: MLP with Ensemble

- How to build an ensemble?
 - **Deep Ensemble**: Train multiple MLPs independently
 - **Packed-Ensemble**: Pack multiple MLPs into one and train them jointly
 - **TabM & TabM_{mini}**: Share most parameters across ensemble members



Parameter-efficient ensembles can outperform
both a single MLP and a conventional deep ensemble

TabM: MLP with Ensemble



Summary: MLP-based Architectures

- **Simple MLPs can be strong models for tabular prediction** when designed and trained carefully, e.g.:
 - Richer representations for numerical and categorical features
 - Better preprocessing and training recipes
 - Neighborhood information from similar training samples
 - Efficient ensembling of multiple predictors

MLP-based models remain a strong/flexible direction for tabular data

Next: Attention-based Architectures

What is Attention?

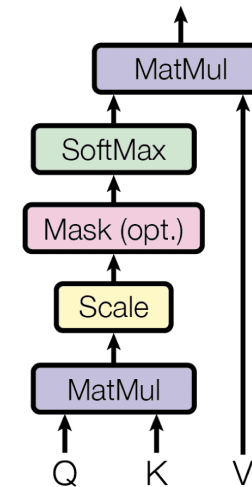
- Attention **identifies relevant relationships among items** and **aggregates information** accordingly
 - Attention can flexibly **capture interactions among items**
 - Attention has been **widely used across diverse domains**, including text, images, time-series, and so on

- **Scaled dot-product attention**

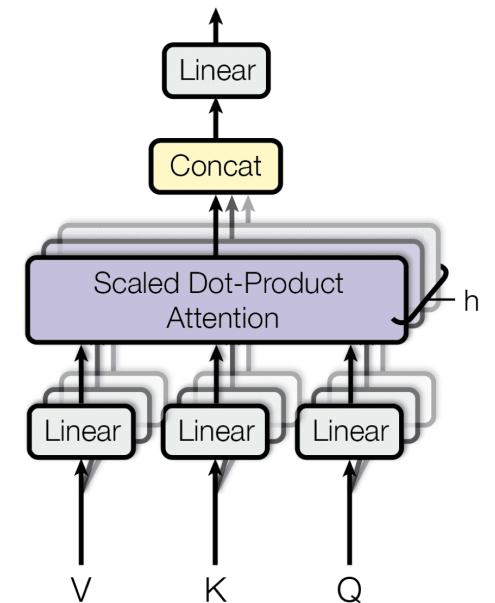
$$\mathbf{o}_i = \sum_j w_{ij} \mathbf{v}_j, \quad w_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d})}{\sum_k \exp(\mathbf{q}_i^\top \mathbf{k}_k / \sqrt{d})}.$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right) \mathbf{V}$$

Scaled Dot-Product Attention

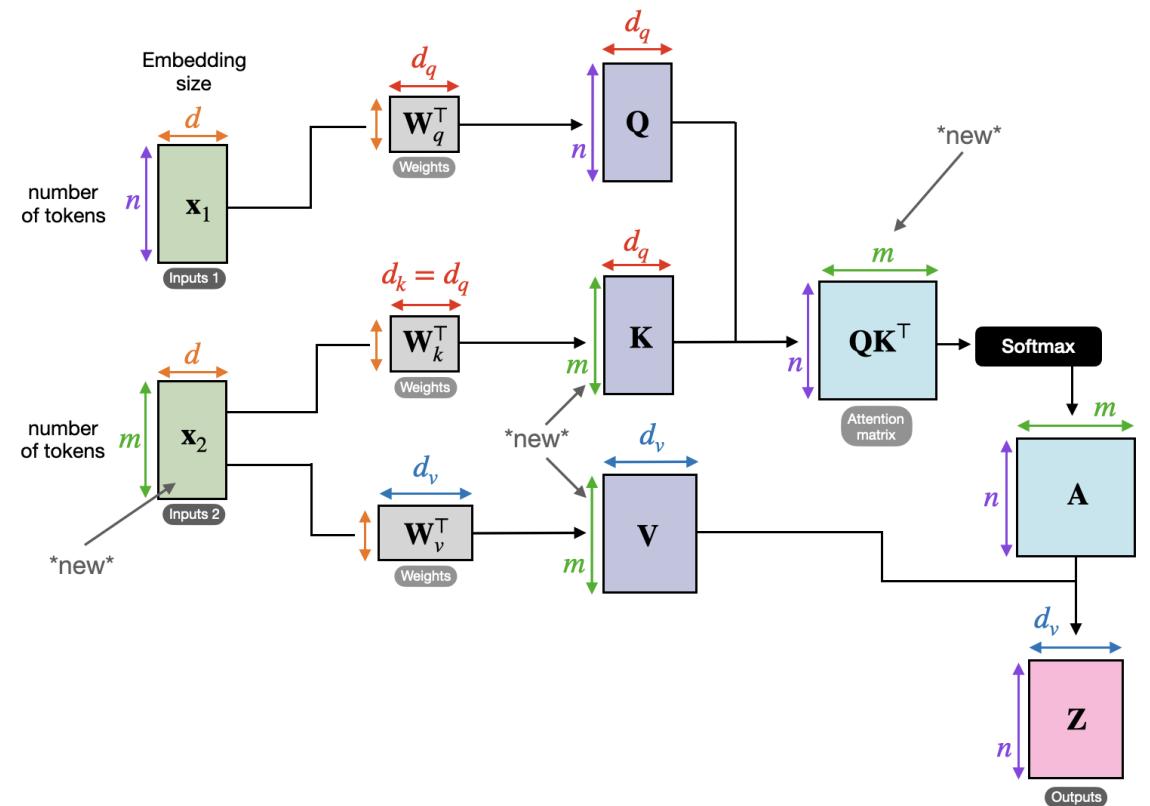
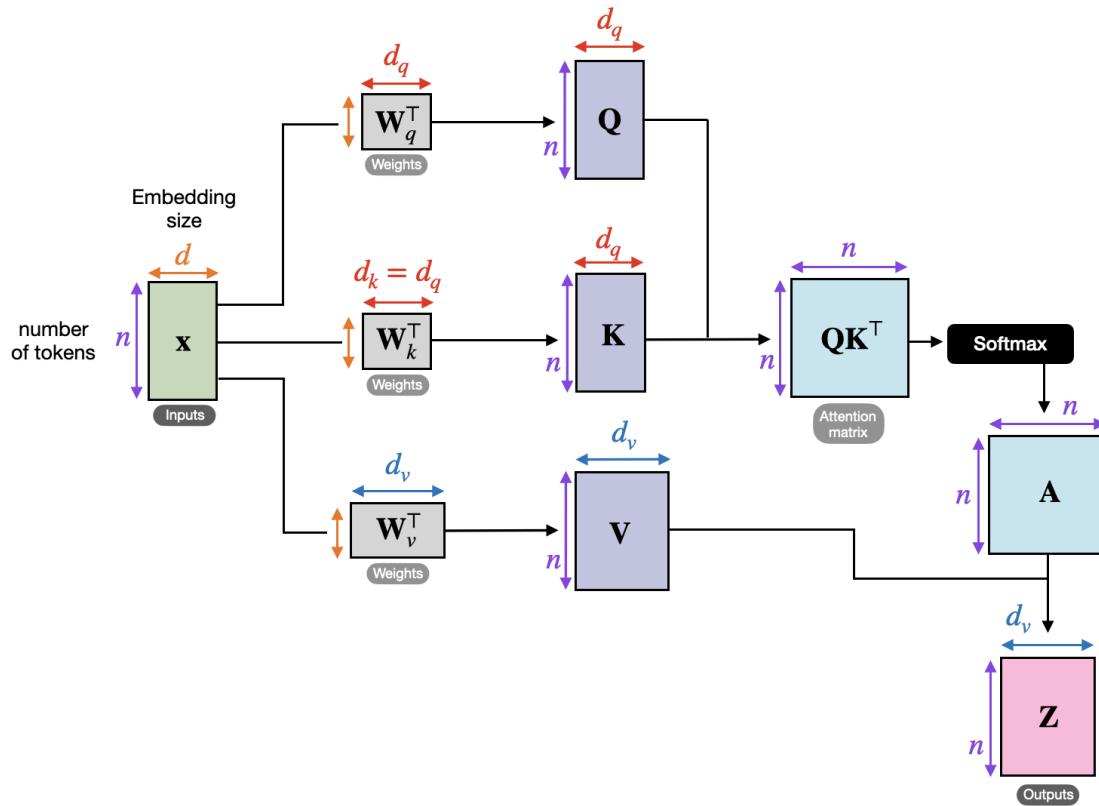


Multi-Head Attention



What is Attention?

- Self-Attention and Cross-Attention

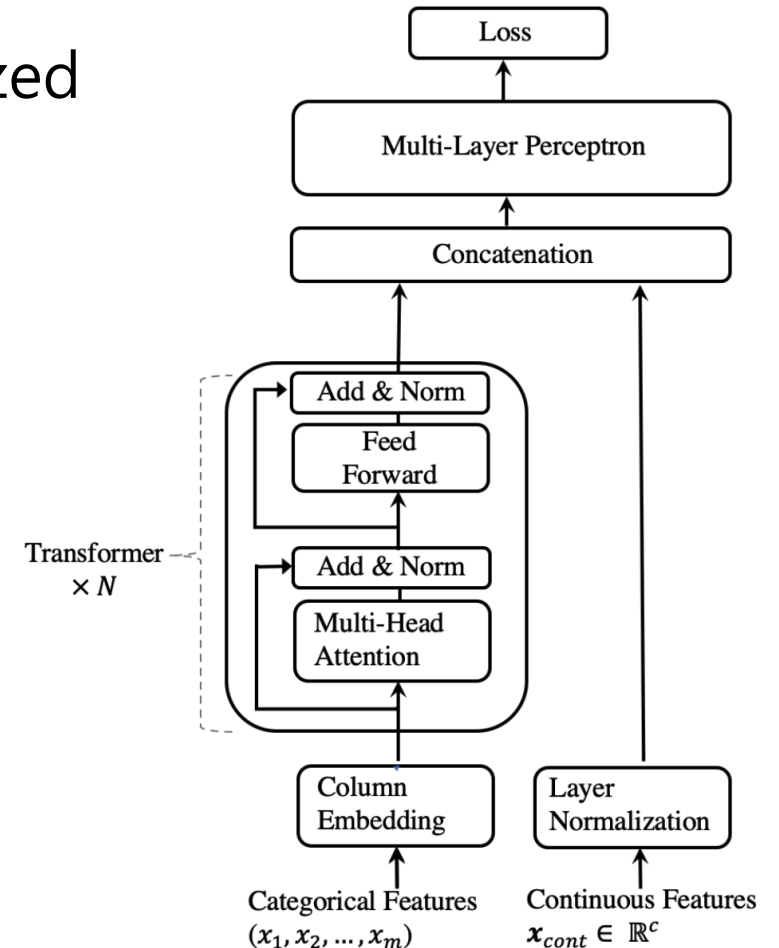


Why Attention for Tabular Data?

- Tabular prediction often **depends on interactions among columns**
 - E.g., "Debt" becomes meaningful when considered together with "Income"
- A tabular row can be **represented as a set of column tokens**
 - Numerical and categorical features are embedded into vectors
 - Attention allows each feature token to incorporate information from other relevant features
- Attention operation is **permutation-equivariant**
 - Tabular columns have no natural order
 - Reordering feature tokens should not change the prediction

TabTransformer

- **Core idea:** Treat each categorical column as a token, and pass these tokens through a Transformer
 - Each categorical embedding becomes contextualized by all other categorical values
- What about **numerical features**?
 - Numerical columns are kept as scalars
 - They are concatenated with the contextual categorical embeddings before the MLP head



FT-Transformer

- FT-Transformer = Feature Tokenizer + Transformer

- Feature Tokenizer

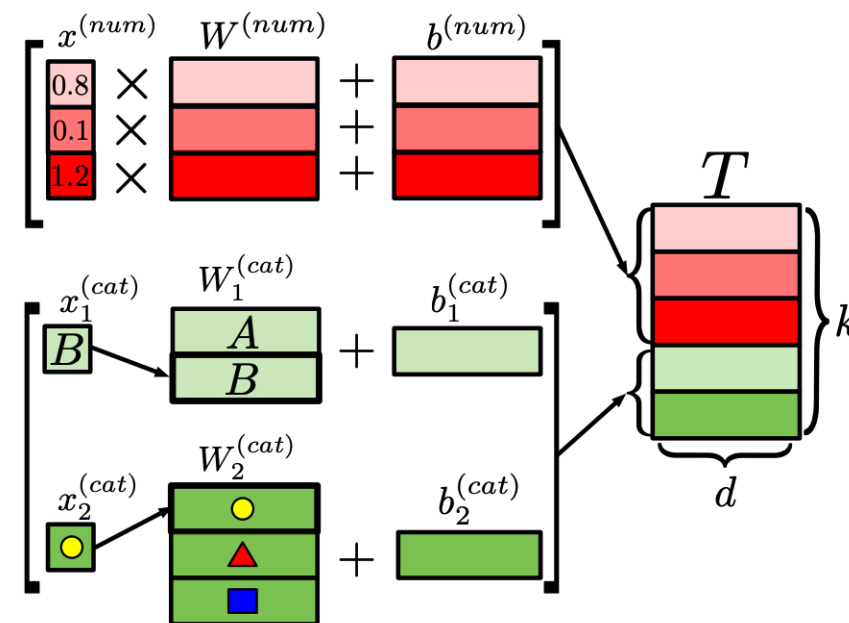
- This transforms the input features to **embeddings** $T \in \mathbb{R}^{k \times d}$

$$T_j = b_j + f_j(x_j) \in \mathbb{R}^d \quad f_j : \mathbb{X}_j \rightarrow \mathbb{R}^d.$$

- Numerical features are linearly transformed
- Categorical features are mapped through the lookup table

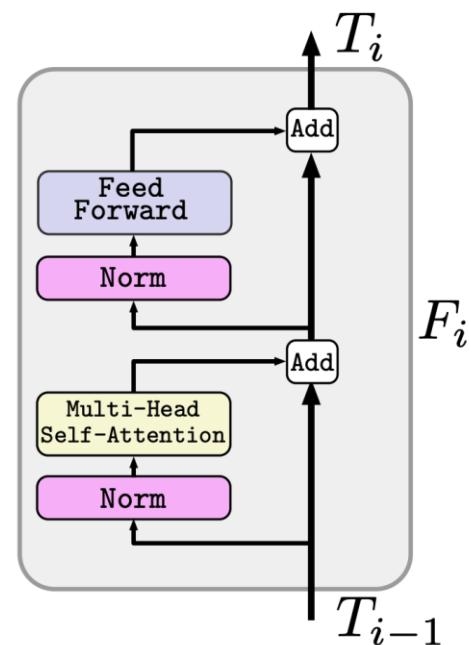
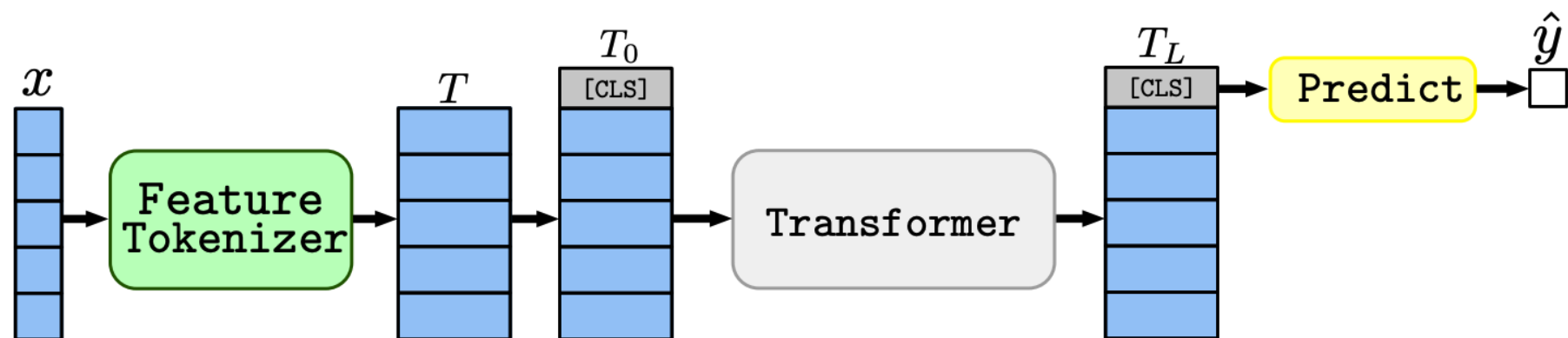
$$T_j^{(num)} = b_j^{(num)} + x_j^{(num)} \cdot W_j^{(num)}$$

$$T_j^{(cat)} = b_j^{(cat)} + e_j^T W_j^{(cat)}$$



FT-Transformer

- FT-Transformer = Feature Tokenizer + Transformer
- Transformer
 - Append [CLS] token to the embedding sequence
 - Apply **Transformer layers** with PreNorm

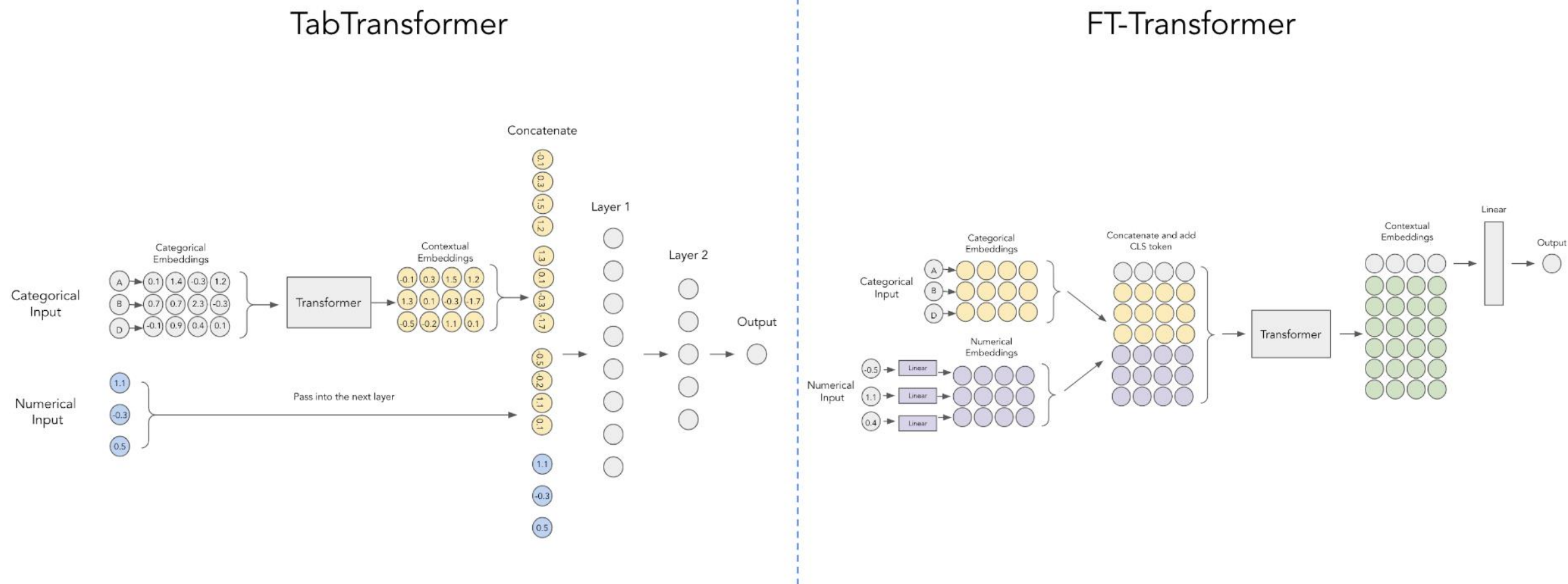


- The final representation of the [CLS] token is used for prediction

$$\hat{y} = \text{Linear}(\text{ReLU}(\text{LayerNorm}(T_L^{[\text{CLS}]}))).$$

FT-Transformer

- TabTransformer vs FT-Transformer



FT-Transformer

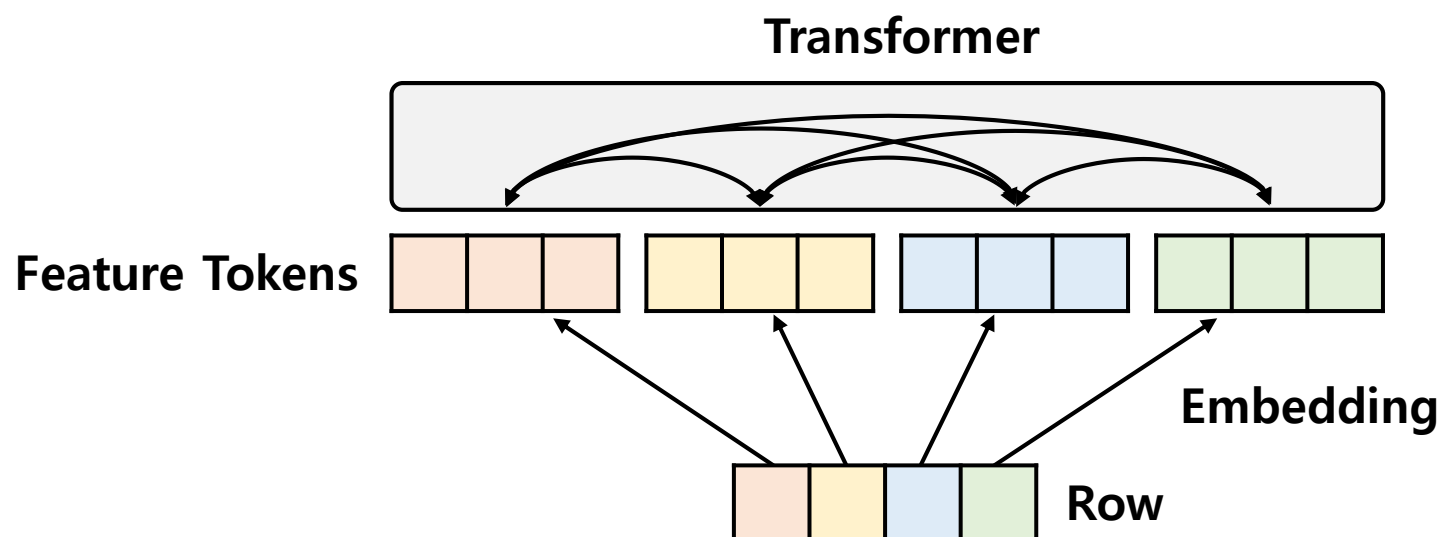
- FT-Transformer outperforms MLP and GBDT baselines

	CA ↓	AD ↑	HE ↑	JA ↑	HI ↑	AL ↑	EP ↑	YE ↓	CO ↑	YA ↓	MI ↓	rank (std)
TabNet	0.510	0.850	0.378	0.723	0.719	0.954	0.8896	8.909	0.957	0.823	0.751	7.5 (2.0)
SNN	0.493	0.854	0.373	0.719	0.722	0.954	0.8975	8.895	0.961	0.761	0.751	6.4 (1.4)
AutoInt	0.474	0.859	0.372	0.721	0.725	0.945	0.8949	8.882	0.934	0.768	0.750	5.7 (2.3)
GrowNet	0.487	0.857	–	–	0.722	–	0.8970	8.827	–	0.765	0.751	5.7 (2.2)
MLP	0.499	0.852	0.383	0.719	0.723	0.954	0.8977	8.853	0.962	0.757	0.747	4.8 (1.9)
DCN2	0.484	0.853	0.385	0.716	0.723	0.955	0.8977	8.890	0.965	0.757	0.749	4.7 (2.0)
NODE	0.464	0.858	0.359	0.727	0.726	0.918	0.8958	8.784	0.958	0.753	0.745	3.9 (2.8)
ResNet	0.486	0.854	0.396	0.728	0.727	0.963	0.8969	8.846	0.964	0.757	0.748	3.3 (1.8)
FT-T	0.459	0.859	0.391	0.732	0.729	0.960	0.8982	8.855	0.970	0.756	0.746	1.8 (1.2)

	CA ↓	AD ↑	HE ↑	JA ↑	HI ↑	AL ↑	EP ↑	YE ↓	CO ↑	YA ↓	MI ↓
Default hyperparameters											
XGBoost	0.462	0.874	0.348	0.711	0.717	0.924	0.8799	9.192	0.964	0.761	0.751
CatBoost	0.428	0.873	0.386	0.724	0.728	0.948	0.8893	8.885	0.910	0.749	0.744
FT-Transformer	0.454	0.860	0.395	0.734	0.731	0.966	0.8969	8.727	0.973	0.747	0.742

FT-Transformer

- FT-Transformer provides a **standard template for attention-based modeling** of tabular data
 - **Treat each column as a feature token**
 - Linearly project numerical features
 - Embed categorical features through lookup tables
 - **Apply self-attention across feature tokens** to model feature interactions

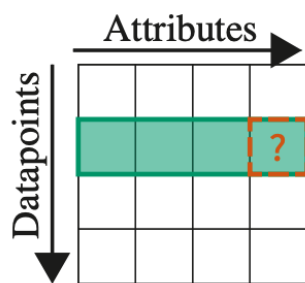


Attention across Data Samples

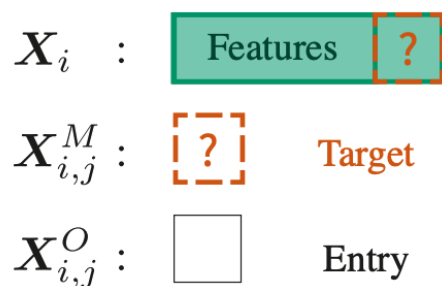
- **Limitation of feature-wise attention:**
FT-Transformer processes each row independently
- However, related samples provide useful information for prediction
 - E.g., nearest-neighbor prediction works well in tabular data
- **Question:** Can a model learn interactions among data samples?
 - E.g., modeling a relationship between sample A and sample B
 - This could be combined with feature-wise attention
 - E.g., sample A's first column $A_1 \leftrightarrow$ sample B's second column B_2

Non-Parametric Transformers (NPTs)

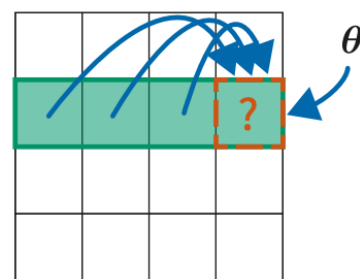
- Prior work: **parametric modeling**
 - Models learn parameters θ from training data $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n$ to maximize the likelihood $p(y|x; \theta)$
 - This does **not consider direct dependencies between datapoints**
- NPTs consider **additional flexibility of using training data directly**
 - Maximize the likelihood $p(y|x, \mathcal{D}_{\text{train}}; \theta)$



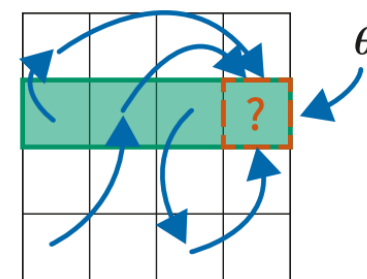
(a) Input Data



(b) Notation



(c) Parametric Model

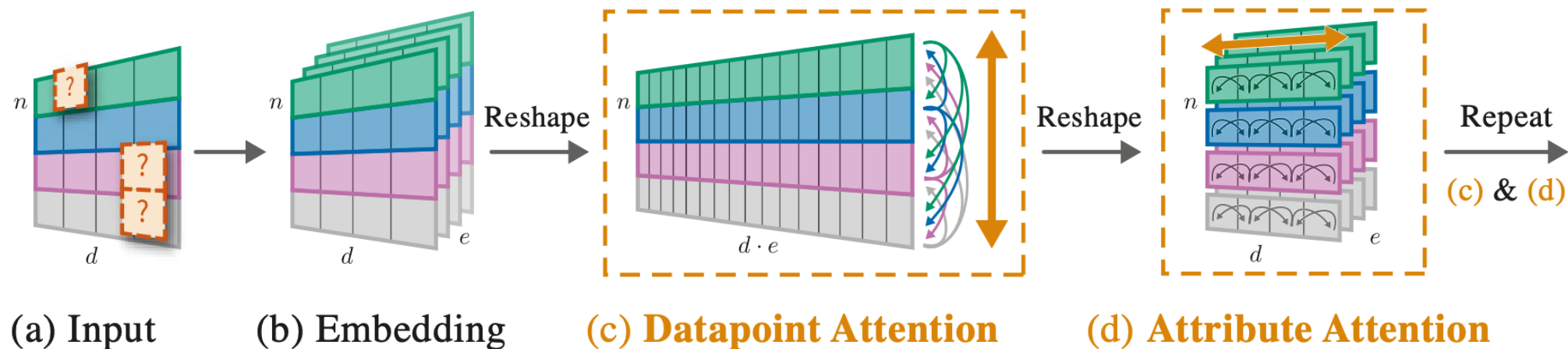


(d) NPT

Non-Parametric Transformers (NPTs)

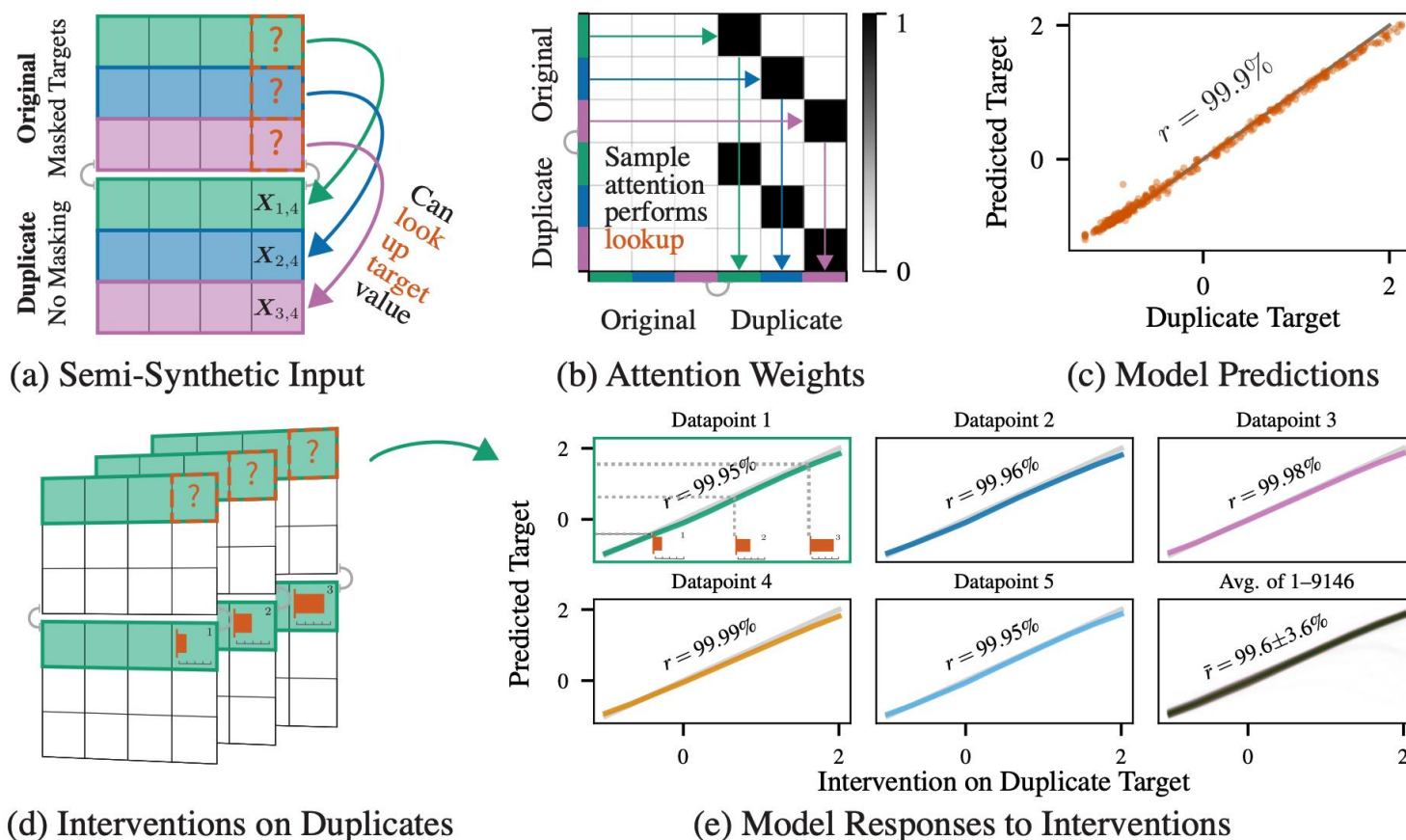
- **NPT Architecture:**

- NPTs take as input the entire dataset and masking matrix
 - Learn to predict masked data from unmasked data such as masked language modeling
- Two types of attention:
 - Attention Between Datapoints – flatten and apply multi-head self-attention (MHSA)
 - Attention Between Attributes – apply MHSA independently to each row



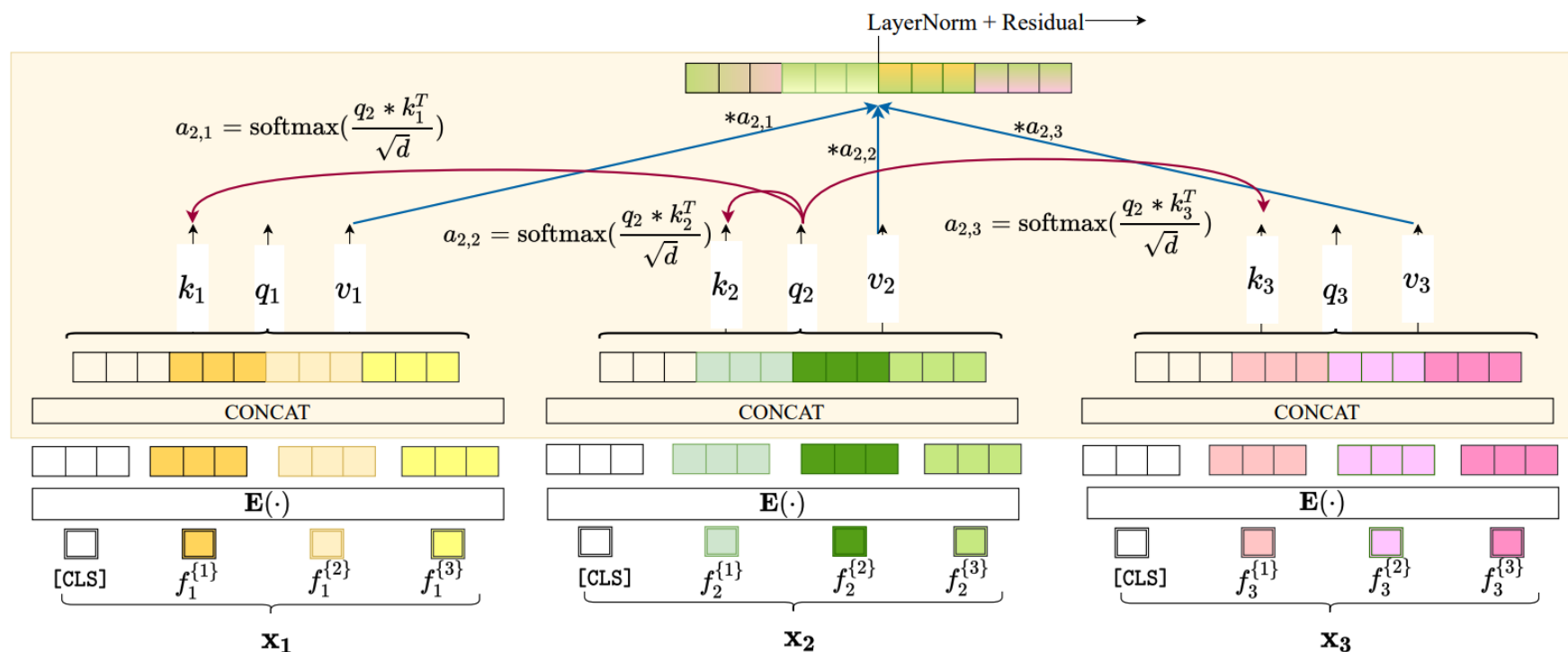
Non-Parametric Transformers (NPTs)

- **Empirical Analysis**
 - NPTs can learn to predict using attention between datapoints



SAINT

- **Self-Attention** and **Intersample Attention** Transformer (SAINT)
 - Similar to NPTs – Feature-wise self-attention & Intersample attention
 - **Additional contribution:** Contrastive and denoising pre-training for label-limited settings



Attention across Data Samples

- **Main Idea**

- Allow samples to interact with other samples during prediction via attention
- Related rows can provide useful contextual information

- **Strengths**

- Captures **sample-level relationships**
- Acts like a learned, flexible version of **nearest-neighbor reasoning**
- Useful when similar samples provide complementary evidence

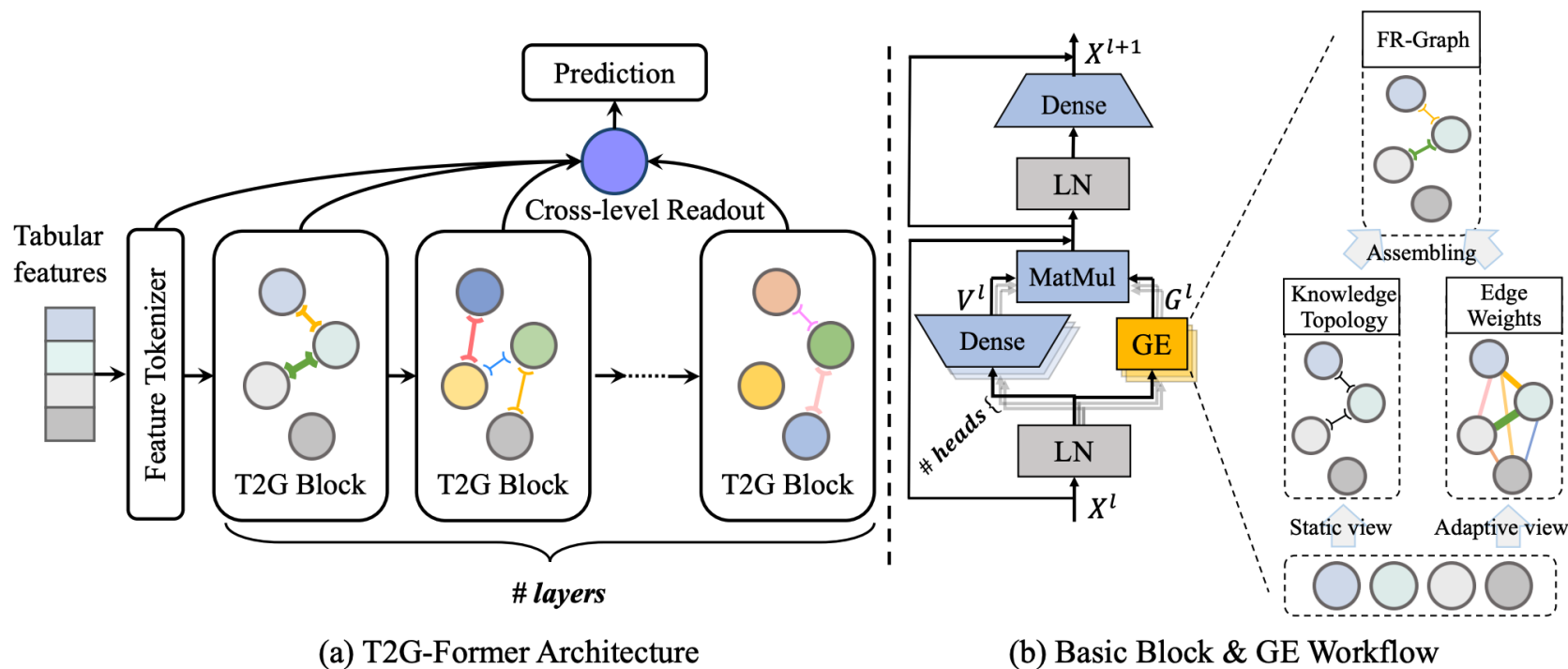
- **Limitations**

- Higher computational and memory cost
- Requires care to avoid target leakage

Exploiting Better Structure for Interactions

Recent architectures **redesign attention to better model feature interactions** in tabular data

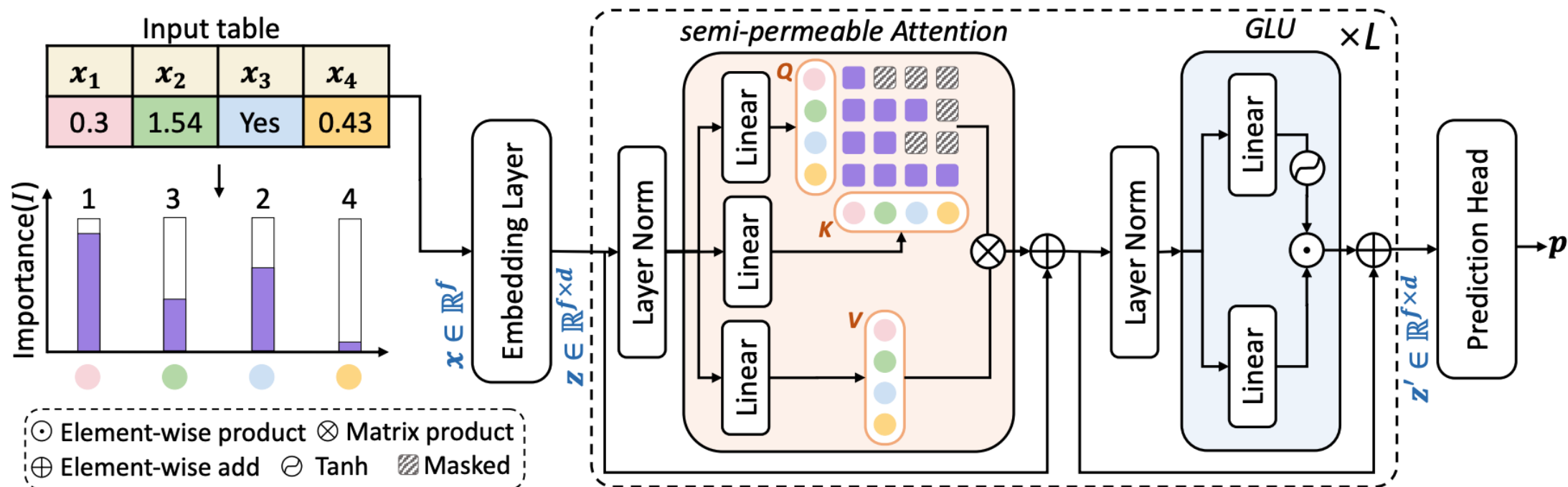
- **T2G-Former**: Guide feature interactions using a relation graph



Exploiting Better Structure for Interactions

Recent architectures **redesign attention to better model feature interactions** in tabular data

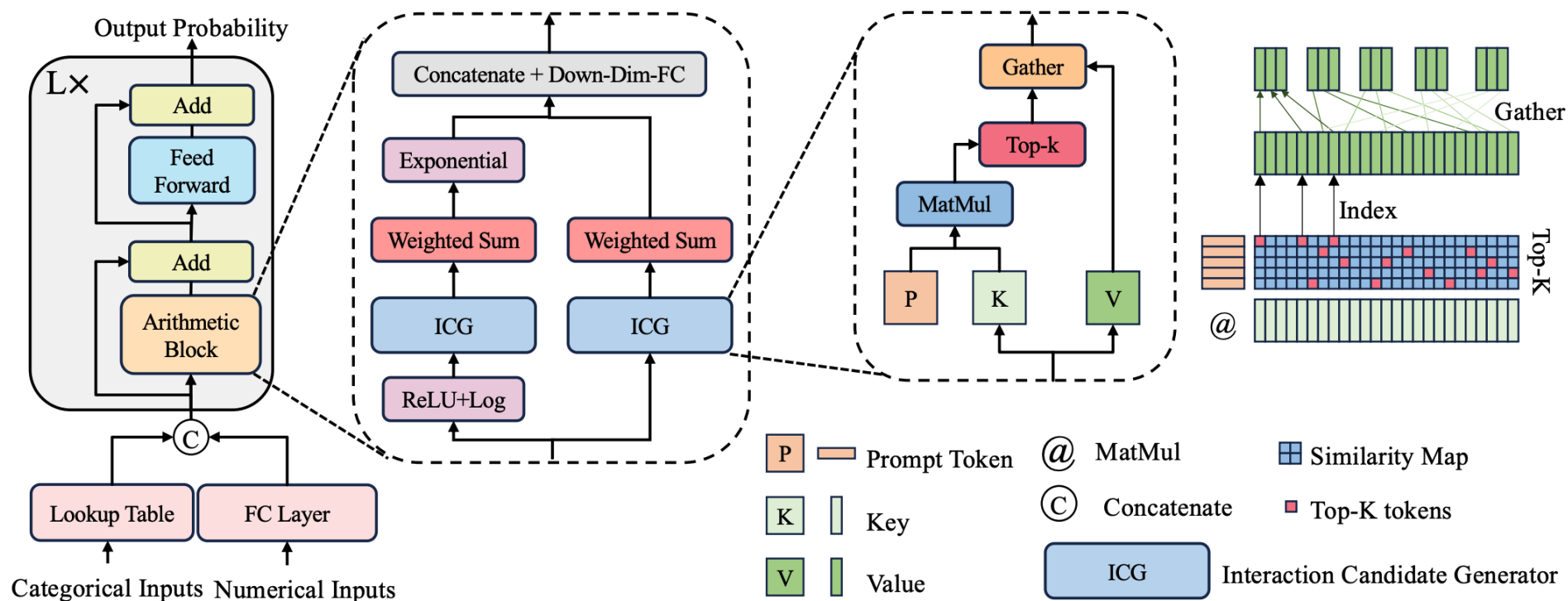
- **ExcelFormer**: Restrict harmful information flow between features



Exploiting Better Structure for Interactions

Recent architectures **redesign attention to better model feature interactions** in tabular data

- **AMFormer**: Model arithmetic feature interactions explicitly



Summary: Attention-based Architectures

- **Attention-based models focus on learning interactions**
 - Feature-level interactions within each row
 - Sample-level interactions across rows
 - Structured or constrained interactions for tabular data
- **Recent models improve plain self-attention by asking**
 - Which features should interact?
 - Which samples should interact?
 - Which interactions should be restricted or emphasized?

Attention gives tabular models a flexible way to model relationships among features and samples

Summary

- Tabular deep learning is **not only about deeper networks**
- **Tree-based models remain strong baselines**
- Deep models become **attractive when representation learning, interactions, retrieval, or extensibility matter**
- The key is **designing the right inductive bias** for tabular data

Thank You for Your Attention!
